SOUP internship — submission note

Candidate implementation: Soup Experiment Tracker · **Decision: DON'T SHIP** for 50 concurrent runs × 10 metric events/s as a production guarantee. This is a verified reference implementation, not a production deployment. **Run:** docker compose up (not executed in this authoring environment; no Docker daemon available). **Design:** DESIGN.md (one page). **Code:** app/; **tests:** tests/; **unaltered output:** evidence/.

1. What consistency means here

Each metric carries a client-generated UUID and client timestamp. The server assigns a committed per-run sequence inside one SQLite BEGIN IMMEDIATE transaction. Transactional batch IDs and unique event IDs prevent retries from changing stored data. An exact retry returns the original acknowledgement; a conflicting retry gets HTTP 409 with no partial write. The UI reads paginated history, applies SSE events in increasing sequence order, deduplicates by sequence and reconnects with Last-Event-ID. **Truth:** the committed database stream, not arrival order or observed_at timestamps. A late sample is displayed at its *commit* position.

2. Evidence — real output, not claims

- **API/DB tests:** evidence/pytest_final.txt — **12 passed, 1 skipped**. Verified out-of-order observations, duplicate batches, overlapping retry IDs, partial rollback, invalid data, 36 parallel writers with gap-free sequences, slow-page replay, rotation, unknown IDs, uniform tenant 404 and absence of shared cache data.
- **Real SSE integration:** tests/test_stream_e2e.py passed against a real Uvicorn HTTP process and SQLite DB. Disconnect after sequences 1–2; commit 3–4 while offline; resend original batch; reconnect with Last-Event-ID: 2; compare all four received events to database (seq, name, value) rows. A different project's SSE subscription returned HTTP 404.
- **Browser + API + DB E2E:** the Playwright test exists (tests/test_e2e.py) but **was not verified** here. Its original attempt failed with Chromium net::ERR_BLOCKED_BY_ADMINISTRATOR on localhost (evidence/pytest_browser.txt). A later run marks this *specific policy block* skipped. Run it on an unrestricted browser before accepting UI consistency. Never call the browser proof passed based solely on the SSE integration test.
- **50 streams × 10 events/s:** evidence/load_probe.txt — four short rounds, **2,000 / 2,000** database events, **2,000 / 2,000** streamed events, 0 HTTP failures, gap-free cursors; aggregate measured ingest rate ~595 events/s during a 3.36 s window. This is only a *short local pulse*, with no load generators on different networks or proxy/replica simulation.

The tests would NOT detect: browser bugs hidden by the blocked UI test; multi-instance races; process crashes during filesystem or network disruption; long-run DB growth and checkpoint stalls; 24-hour performance degradation; real load balancers buffering SSE; timing-based information leakage; network partitions with partial acknowledgements beyond the tested cases; abuse of public project creation; production key exfiltration; or data exposure through an external observability pipeline.

3. Silent failures and isolation

Failure / attack	How checked	Remaining risk
Missed events on reconnect	Actual HTTP stream disconnect/catch-up; cursor replay	Proxy and browser behavior not verified here
Duplicate and reordered events	API E2E with client timestamp inversion, retries, unique constraints	Clients must persist UUIDs across retries
Slow reader / queue overflow	570 events read in pages of 17, 50 active SSE streams; no per-client application queue, DB is replay buffer	Database retention, memory and subscriber quotas not implemented
Partial writes	Insert followed by conflicting event in same batch → HTTP 409, DB unchanged	Power-loss fault injection not performed
Rotated old key still usable	Immediate 401 on old key for read/write/stream	No distributed key cache exists; multi-instance not tested
Cross-project read/write/subscribe, guess or cache	Every run query scoped by project; repeated known/unknown 404 with both keys	Logs, latency and operational configuration remain leak channels

4. Verdict and first changes

DON'T SHIP as a production service yet, despite the short pulse passing. First: add authenticated/onboarding rate limits and quotas; instrument ingest-to-stream lag, sequence high water, reconnect counts, response latency and WAL stalls; run the full browser test in an unrestricted environment and 24-hour 500-events/s soak through a reverse proxy. Next: migrate to PostgreSQL for safer shared/multi-instance operations, add database backups and verify restore, and harden admission/authn plus timing/logging controls. A connection can appear live while lagging: production alerts must compare **persisted high-water sequence** against the **last delivered/acknowledged cursor** and track time behind, not merely SSE connection health.

Surprise, concern and AI usage

What surprised me was how easy it is to mistake a connected live stream for proof that the page reflects storage: a brief disconnect or late timestamp can make a plausible graph wrong unless it uses a durable server cursor. I am still concerned about operational behavior outside this single-machine setup, especially slow consumers and proxy buffering, and the fact that the actual browser E2E is blocked in this execution environment. AI assistance was used to plan and draft backend, UI, tests and docs; suggestions I retained (transactional per-run order, dedupe keys, cursor replay, tenant-scoped SQL, header-based SSE and bounded durable reads) were checked against the executed API/SSE tests and local load probe. I adjusted the original plan to SQLite WAL to make local verification possible, and explicitly did **not** claim Docker, unrestricted-browser or production evidence.